

Universidad del Valle de Guatemala



Análisis y Diseño de Algoritmos

Examen Parcial #2 — Solución desarrollada

Nicolás Concuá - 23197

Docente: Tomás Gálvez P.

Semestre 1, 2026

Repositorio GitHub:

<https://github.com/nicoCT04/Parcial2AED>

Video YouTube (no listado):

[Inserte aquí enlace al video]

Guía de la solución

Para cada problema se presenta:

- (a) propiedades identificadas,
- (b) demostración formal de dichas propiedades siguiendo los procedimientos vistos en clase (decisiones, subproblemas, necesaria optimalidad / herencia e intercambio / greedy-choice property),
- (c) el pseudocódigo del algoritmo correspondiente

Problema 1: Hacer Sencillo (Coin Change)

Enunciado: dado un monto m , alcanzarlo usando la menor cantidad de monedas posible con denominaciones $D = \{1, 5, 10, 25\}$. Se puede tomar tantas monedas como se necesite de cada denominación.

(a) Propiedades seleccionadas

- ☒ **Subestructura óptima**
- ☒ **Subproblemas traslapados**
- ☒ **Greedy-choice property**
- ☐ Se puede modelar como una matroide ponderada

Justificación de la selección

Marcamos **subestructura óptima** porque al elegir una moneda d del conjunto D para alcanzar un monto m , el problema se reduce a hacer sencillo $m - d$ con las mismas denominaciones — un subproblema de la misma forma pero menor — y la solución óptima al original requiere obligatoriamente la solución óptima al subproblema (lo demostramos formalmente en el inciso (b)).

Marcamos **subproblemas traslapados** porque la recurrencia $C(m) = 1 + \min\{C(m - d) : d \in D, d \leq m\}$ produce un árbol de recursión donde el mismo subproblema $C(k)$ aparece por múltiples caminos (e.g., $C(20)$ se alcanza desde $C(30)$ tanto eligiendo 5 como 10), lo que justifica que admite también una solución por programación dinámica.

Marcamos **greedy-choice property** porque el sistema canónico $\{1, 5, 10, 25\}$ permite que la elección local de la moneda más grande disponible siempre forme parte de una solución óptima — lo demostramos por intercambio en el inciso (b), analizando los rangos de m . Esto nos permite resolver el problema con un acercamiento greedy puro (más eficiente que DP).

Por otro lado, **NO marcamos matroide ponderada** porque una matroide se define como una pareja (S, I) donde S es el conjunto de objetos del problema e I es la familia de subconjuntos independientes (con herencia e intercambio). En este problema la solución no es un subconjunto de monedas distintas, sino un **multiconjunto** (las monedas se repiten libremente: para Q2.93 usamos 11 veces la de 25). No hay un conjunto base discreto sobre el que definir independencia, ni propiedad de intercambio entre subconjuntos sin repetición. Por tanto, el modelo de matroide simplemente no aplica al problema, aunque el paradigma greedy sí funcione por otra razón (el sistema canónico de denominaciones).

(b) Demostración de las propiedades

B.1 Subestructura óptima

Siguiendo los tres pasos vistos en clase para identificar y demostrar subestructura óptima:

1) Decisiones. En cada paso decidimos qué denominación de moneda $d \in D$ agregamos a la solución, eligiendo de entre las que no excedan el monto restante.

2) Subproblemas. Al elegir una moneda d para alcanzar un monto m , queda el subproblema de hacer sencillo el monto $m - d$ con la mínima cantidad de monedas posibles de las mismas denominaciones D . Es la misma forma del problema original, pero con menor entrada.

3) Necesaria optimalidad (por contradicción). Sea $C(m)$ el número mínimo de monedas para el monto m , y supongamos que la solución óptima incluye una moneda d , dejando un sub-monto $m - d$. Llamemos $C^*(m - d)$ al número óptimo de monedas para $m - d$, de forma que $C(m) = 1 + C^*(m - d)$.

Si la sub-solución para $m - d$ dentro de $C(m)$ no fuera óptima, existiría $C'(m - d) < C^*(m - d)$, lo que produciría una solución para m con $1 + C'(m - d) < 1 + C^*(m - d) = C(m)$ monedas — contradicción con la optimalidad de $C(m)$. Por tanto, la sub-solución debe ser óptima, lo que demuestra subestructura óptima.

B.2 Subproblemas traslapados y relación de recurrencia

Apoyándonos en la subestructura óptima del paso anterior, planteamos la siguiente relación de recurrencia formal:

$$C(m) = \begin{cases} 0, & \text{si } m = 0 \\ 1 + \min \{ C(m - d) : d \in D, d \leq m \}, & \text{si } m > 0 \end{cases}$$

Si calculáramos esta recurrencia de forma puramente recursiva (sin memoización), el árbol de recursión repetiría subproblemas: por ejemplo, para $m = 30$, llegamos a $C(20)$ tanto desde $C(30) \rightarrow C(25)$ (eligiendo 5) como desde $C(30) \rightarrow C(20)$ (eligiendo 10). El subproblema $C(20)$ aparece por múltiples caminos en el árbol, por lo que **los subproblemas se traslapan**. Esto justifica que el problema también admite solución por programación dinámica (memoización o tabulación), aunque por la greedy-choice property que demostramos a continuación, no hace falta recurrir a DP.

B.3 Greedy-choice property

Procedimiento de selección

Definimos el procedimiento de selección codicioso como: *en cada paso, tomar la moneda de mayor denominación posible que no exceda el monto restante*. Formalmente, dado un monto restante $m > 0$, elegimos $d^* = \max\{d \in D : d \leq m\}$.

Demostración de la propiedad

Demostración (por contradicción / intercambio). sea m el monto a alcanzar y sea $d^* = \max\{d \in D : d \leq m\}$ la moneda greedy. Supongamos que existe una solución óptima S que **no** contiene a d^* . Demostramos que se puede transformar S en otra solución óptima S' que sí incluye d^* , analizando los rangos del algoritmo:

- **Si $m \geq 25$:** $d^* = 25$. Si S no usa monedas de 25, debe alcanzar al menos 25 en valor con monedas de $\{1, 5, 10\}$. Cualquier combinación de éstas que sume 25 usa al menos 3 monedas (por ejemplo, $10+10+5 = 3$ monedas, o $10+5+5+5 = 4$, etc., siendo el mínimo $10+10+5$ con 3 monedas). Reemplazando esas 3+ monedas por una sola de 25 obtenemos una solución S' con menor o igual cantidad de monedas — contradiciendo la optimalidad de S a menos que S ya incluyera 25.

- Si $10 \leq m < 25$: $d^* = 10$. Para alcanzar al menos 10 sin usar la moneda de 10, se requieren al menos 2 monedas (5+5) o más. Reemplazar 5+5 por una sola moneda de 10 reduce el conteo, contradicción.
- Si $5 \leq m < 10$: $d^* = 5$. Sin la moneda de 5 se requieren 5 monedas de 1 para alcanzar 5 — reemplazar por una sola moneda de 5 reduce el conteo.
- Si $m < 5$: sólo cabe la moneda de 1, que es la greedy.

En todos los casos, no incluir la moneda greedy d^* permite construir una solución estrictamente mejor (o igual), contradiciendo la optimalidad de la solución que la rechaza. Por tanto, existe una solución óptima que incluye d^* , y combinada con la solución óptima al subproblema $m - d^*$ produce la solución óptima del problema general — **greedy-choice property demostrada**.

Nota crítica: esta propiedad depende de las denominaciones específicas. Para conjuntos distintos como $\{1, 3, 4\}$, greedy fallaría (e.g., para $m = 6$: greedy daría $4+1+1 = 3$ monedas, pero el óptimo es $3+3 = 2$ monedas). Aquí funciona porque $\{1, 5, 10, 25\}$ es un sistema canónico.

(c) Pseudocódigo (paradigma greedy)

Problema No.1 Pseudocódigo

Hacer_sencillo(m, D):

sort(D , descendente)
resultado = []

Para cada d en D

si $d \leq m$:

cantidad = floor(m/d)

resultado.append($(d, cantidad)$)

$m = m - cantidad * d$

si $m == 0$

break

return resultado

Complejidad: $O(|D|)$ — lineal en el número de denominaciones ($|D| = 4$ aquí, así que es $O(1)$ en la práctica).

Problema 2: Knapsack Fraccionado

Enunciado: una bolsa que aguanta W unidades de peso, y n artículos donde el artículo i tiene precio p_i (sobre el total disponible) y hay w_i unidades disponibles. Es posible tomar porciones de cada artículo. Maximizar el valor total robado.

(a) Propiedades seleccionadas

- ☒ **Subestructura óptima**
- ☐ Subproblemas traslapados
- ☒ **Greedy-choice property**
- ☐ Se puede modelar como una matroide ponderada

Justificación de la selección

Marcamos **subestructura óptima** porque al decidir cuánto tomar de un artículo, queda el subproblema de optimizar el espacio restante $W - c_i$ con los artículos y cantidades que sobran — la misma estructura del problema original con menor entrada — y la ganancia óptima total exige la ganancia óptima del subproblema (lo demostramos en el inciso (b)).

Marcamos **greedy-choice property** porque al ordenar los artículos por valor por unidad de peso $v_i = p_i / w_i$ descendientemente, tomar la mayor cantidad posible del artículo con v más alto siempre forma parte de una solución óptima. La demostración (inciso (b)) usa el argumento clásico: si una solución no toma la cantidad greedy completa del artículo de mayor valor unitario, podemos intercambiar parte de los artículos de menor valor unitario por el de mayor valor unitario sin perder ganancia, contradiciendo que la otra solución fuera óptima. La naturaleza **fraccionaria** del problema garantiza que siempre podemos llenar exactamente el espacio (no quedan huecos).

En contraste, **NO marcamos subproblemas traslapados**. Aunque uno podría plantear una recurrencia, la presencia de la greedy-choice property elimina la ramificación: en cada paso hay una sola decisión correcta (tomar el de mayor valor unitario), por lo que no hay árbol de recursión con ramas alternas que evaluar — sólo una secuencia lineal de subproblemas. No hay re-cálculo de los mismos subproblemas porque no hay caminos alternativos. Esto es una diferencia clave con la versión *0/1 knapsack*, donde sí hay subproblemas traslapados (porque para cada artículo hay que decidir entre incluirlo o no, ramificando el árbol).

Tampoco marcamos **matroide ponderada**. Una matroide define independencia sobre un conjunto base discreto S — la pertenencia a un subconjunto es binaria (un objeto está o no está). Aquí la solución es una **asignación de cantidades fraccionarias** ($c_1, c_2, \dots, c_n$) con valores reales en $[0, w_i]$, no una elección binaria. La propiedad de intercambio de matroides ("si $|A| < |B|$, existe $x \in B \setminus A$ tal que $A \cup \{x\} \in I$ ") no tiene una traducción natural a porciones reales de objetos. El problema sí es greedy, pero por la estructura fraccionaria, no por una matroide subyacente.

(b) Demostración de las propiedades

B.1 Subestructura óptima

Siguiendo los tres pasos vistos en clase para identificar y demostrar subestructura óptima:

1) Decisiones. Visualizamos la solución como $(c_1, c_2, \dots, c_n)$, donde c_i es la cantidad robada del artículo i . La decisión es cuánto de qué artículo agregamos a la mochila.

2) Subproblemas. Luego de elegir un artículo y la cantidad por tomar, queda el subproblema de optimizar el espacio restante $W - c_i$ en la mochila con los artículos restantes y sus cantidades disponibles.

3) Necesaria optimalidad (por contradicción). Supongamos que la solución óptima es $(c_1, \dots, c_n)$ con ganancia G . Al quitar c_n queda la pregunta: ¿cuál es la mejor combinación que ocupa el peso $W_o = W - c_n$ con los $n - 1$ artículos restantes? Sea G_o la ganancia óptima para ese subproblema.

Si en la solución "óptima" original esa sub-solución no fuera G_o sino una $G_s < G_o$, entonces la ganancia total sería $G_s + p(c_n) < G_o + p(c_n)$, contradiciendo que era óptima. Por tanto, la sub-solución debe ser óptima — **subestructura óptima demostrada.**

B.2 Greedy-choice property

Procedimiento de selección

Definimos el procedimiento de selección codicioso como: *elegir el artículo con mayor valor por unidad de peso $v_i = p_i / w_i$ y tomar la cantidad máxima posible (toda la disponible o lo que quepa en la mochila, lo que sea menor)*. Repetimos hasta llenar la mochila o agotar los artículos.

Demostración de la propiedad

Demostración (por contradicción / intercambio). Ordenemos los artículos descendientemente por valor por unidad de peso, así $v_n \geq v_{n-1} \geq \dots \geq v_1$. Sea c_n la cantidad greedy del artículo n (la de mayor valor unitario).

Supongamos una solución "óptima" que toma k_n con $0 \leq k_n < c_n$ del artículo n . El espacio $c_n - k_n$ debe llenarse con otro artículo j (con $v_j \leq v_n$) para alcanzar el peso W (de lo contrario, queda espacio vacío y la solución no es óptima).

Como $v_n \geq v_j$, podemos reemplazar el peso $c_n - k_n$ que tomamos del artículo j por el mismo peso del artículo n , ganando:

$$(c_n - k_n) \cdot v_n \geq (c_n - k_n) \cdot v_j$$

lo que produce una ganancia mayor o igual a la presunta óptima — contradiciendo que $k_n < c_n$ en la óptima. Por tanto, la decisión greedy de tomar c_n del artículo de mayor valor unitario forma parte de una solución óptima. **Greedy-choice property demostrada.**

(c) Pseudocódigo (paradigma greedy)

Problema No. 2 Pseudocódigo

Knapsack_fracionado (items, w):

// items[i] = (precio_total p_i, peso_disponible w_i)

// Calcular Valor por Unidad $v_i = p_i / w_i$

Para cada item i en items:

 item.v = item.p / item.w

// ordenar desc por v

Sort(items, descendente por v)

espacio_restante = w

Valor_total = 0

solución = [] // (item, cantidad tomada)

Para cada item en items:

 si espacio_restante == 0:

 break

 Cantidad = min(item.w, espacio_restante)

 solución.append((item, cantidad))

 Valor_total += Cantidad * item.v

 espacio_restante -= cantidad

return (valor_total, solución)

Complejidad: $O(n \log n)$ por el ordenamiento + $O(n)$ por el recorrido = **$O(n \log n)$** .

Problema 3: Combinaciones en teclado Nokia 3230

Enunciado: contar cuántas combinaciones de n dígitos pueden formarse comenzando en cualquier dígito, donde cada dígito siguiente debe ser adyacente (arriba, abajo, izquierda, derecha) al anterior en el teclado del Nokia 3230. Las teclas * y # no son válidas.

1	2	3
4	5	6
7	8	9
*	0	#

Adyacencias válidas (incluido el 0):

0 -> {8}	5 -> {2, 4, 6, 8}
1 -> {2, 4}	6 -> {3, 5, 9}
2 -> {1, 3, 5}	7 -> {4, 8}
3 -> {2, 6}	8 -> {0, 5, 7, 9}
4 -> {1, 5, 7}	9 -> {6, 8}

(a) Propiedades seleccionadas

- ☒ Subestructura óptima
- ☒ Subproblemas traslapados
- ☐ Greedy-choice property
- ☐ Se puede modelar como una matroide ponderada

Justificación de la selección

Marcamos **subestructura óptima** porque el conteo total se descompone aditivamente sobre subproblemas bien definidos: si $f(k, d)$ cuenta las combinaciones de longitud k que terminan en el dígito d , entonces $f(k, d)$ depende exclusivamente de los conteos correctos de $f(k - 1, v)$ para cada vecino v de d . Si cualquier subproblema fuera incorrecto, el conteo del problema mayor también lo sería — análogo a la "necesaria optimalidad" en problemas de optimización (lo formalizamos en el inciso (b)).

Marcamos **subproblemas traslapados** porque al expandir la recurrencia $f(k, d) = \sum f(k-1, v)$ sin memoización, el árbol de recursión repite los mismos subproblemas múltiples veces (e.g., $f(2, 5)$ aparece como hijo de $f(3, 2)$, $f(3, 4)$, $f(3, 6)$ y $f(3, 8)$). Sin embargo, sólo hay $O(n \times 10)$ subproblemas distintos, así que con tabulación bottom-up se evalúa cada uno una sola vez. Estas dos propiedades juntas (subestructura óptima + subproblemas traslapados) son exactamente las que definen un problema resoluble por **programación dinámica**.

Por otro lado, **NO marcamos greedy-choice property**. Recordemos que la greedy-choice property es *un criterio de decisión que elige óptimos locales*, y su presencia en un **problema de optimización** garantiza que se puede resolver con greedy. Pero este problema **no es de optimización** — no estamos buscando la "mejor" combinación según algún criterio, sino contando *todas* las combinaciones válidas. No hay óptimo local que elegir: cada combinación válida cuenta exactamente uno y todas son igualmente "buenas". El concepto de greedy simplemente no tiene sentido aquí.

Por la misma razón, **NO marcamos matroide ponderada**. Una matroide ponderada (S, I) es la formalización de problemas de optimización cuya función de peso w nos permite comparar elementos del conjunto base. En este problema no hay función de peso ni elementos a optimizar — sólo hay reglas de adyacencia y un conteo. Tampoco la solución es un "subconjunto independiente": una combinación es una secuencia ordenada con repeticiones (ej. la combinación 22 incluye dos veces el dígito 2), no un conjunto. Ni el dominio ni el objetivo del problema encajan en el modelo de matroide.

(b) Demostración de las propiedades

B.1 Subestructura óptima (estructura recursiva del conteo)

Aunque hablemos de "subestructura óptima" en sentido amplio, en problemas de conteo lo equivalente es que el **conteo total** se compone aditivamente de conteos sobre subproblemas independientes y bien definidos. Seguimos los tres pasos vistos en clase:

1) Decisiones. En el paso k de la construcción, decidimos cuál es el siguiente dígito a marcar (entre los adyacentes al actual).

2) Subproblemas. Definimos $f(k, d)$ = número de combinaciones válidas de exactamente k dígitos que **terminan** en el dígito d . El subproblema es de la misma forma que el problema original, pero con menor longitud.

3) Necesaria optimalidad / aditividad. Sea $T(n)$ el conteo total que buscamos. Toda combinación de longitud n termina en algún dígito $d \in \{0, \dots, 9\}$. Por el principio de la suma, $T(n) = \sum f(n, d)$ (suma sobre $d = 0..9$). Adicionalmente, toda combinación de longitud k que termina en d se construye únicamente a partir de una combinación de longitud $k-1$ que termina en algún vecino $v \in \text{adj}(d)$, agregando el dígito d al final.

Si para algún subproblema $f(k-1, v)$ obtuviéramos un conteo incorrecto, $f(k, d)$ también sería incorrecto — por contradicción, los subproblemas deben resolverse correctamente. Esto demuestra que la solución del problema general se compone **obligatoriamente** de las soluciones correctas a los subproblemas (lo análogo a la necesaria optimalidad).

B.2 Subproblemas traslapados

Si calculáramos $f(k, d)$ recursivamente sin memoización, observemos cómo se repiten subproblemas. Por ejemplo, para calcular $f(4, 5)$ necesitamos $f(3, 2)$, $f(3, 4)$, $f(3, 6)$, $f(3, 8)$. Cada uno de éstos a su vez necesita varios $f(2, \cdot)$:

```
f(3, 2) requiere f(2, 1), f(2, 3), f(2, 5)
f(3, 4) requiere f(2, 1), f(2, 5), f(2, 7)  <- f(2,1) y f(2,5) se repiten
f(3, 6) requiere f(2, 3), f(2, 5), f(2, 9)  <- f(2,3) y f(2,5) se repiten
f(3, 8) requiere f(2, 0), f(2, 5), f(2, 7), f(2, 9) <- f(2,5),(2,7),(2,9) se repiten
```

El subproblema $f(2, 5)$ aparece **en los cuatro** caminos diferentes. Más generalmente, cada nivel del árbol multiplica los subproblemas, pero el conjunto total de subproblemas distintos es $O(n \times 10)$. Sin memoización el árbol crece exponencialmente en n ; con memoización (DP), se evalúa cada subproblema una sola vez. **Subproblemas traslapados confirmados.**

(c) Algoritmo según los 3 pasos de programación dinámica

Siguiendo el procedimiento visto en clase para diseñar un algoritmo de programación dinámica:

Paso 1: Caracterizar la solución óptima

Por la subestructura óptima demostrada en (b), una combinación válida de longitud n que termina en el dígito d se forma extendiendo en una posición una combinación válida de longitud $n - 1$ que termina en algún vecino $v \in \text{adj}(d)$. La solución total es la suma sobre todos los dígitos finales posibles. La caracterización es:

```
f(k, d) = #combinaciones válidas de longitud k que terminan en d
T(n)    = Σ f(n, d)   para d = 0..9  ← solución buscada
```

Paso 2: Plantear la relación de recurrencia

Apoyándonos en la caracterización anterior, planteamos formalmente la relación de recurrencia:

$$f(k, d) = \begin{cases} 1, & \text{si } k = 1 \\ \sum f(k-1, v) & \text{para } v \in \text{adj}(d), \text{ si } k > 1 \end{cases}$$

donde el caso base $f(1, d) = 1$ indica que cada dígito por sí solo forma una combinación válida de longitud 1. La adyacencia $\text{adj}(d)$ incluye al propio dígito d (consistente con que el enunciado lista 00, 11, ..., 99 en sus 36 combinaciones para $n = 2$).

Paso 3: Solución apoyada en la relación de recurrencia (bottom-up)

Implementamos la recurrencia con tabulación **bottom-up** usando una tabla $dp[k][d]$ que almacena los valores de $f(k, d)$. Esto evita el re-cálculo de los subproblemas traslapados identificados en (b):

Problema No. 3 Pseudocódigo

Nokia - combinaciones (n):

// solo dígitos (0-9; y # excluidos)

// propio dígito (consistente con n=2 → 36 combinaciones)

adj = { 0 : [0, 8] 5 : [2, 4, 5, 6, 8]
 1 : [1, 2, 3, 4] 6 : [3, 5, 6, 9]
 2 : [1, 2, 3, 5] 7 : [4, 7, 8]
 3 : [2, 3, 6] 8 : [0, 5, 7, 8, 9]
 4 : [1, 4, 5, 7] 9 : [6, 8, 9]
 3

// dp[k][d] = combinaciones de longitud k que terminan en d
let dp[1...n][0...9]

// Caso base: $f(1, d) = 1$ para todo d

para d = 0 hasta 9:

dp[1][d] = 1

// Llenado bottom up siguiendo la recurrencia

para k = 2 hasta n:

para d = 0 hasta 9

dp[k][d] = 0

para cada v en adj[d]:

dp[k][d] += dp[k-1][v]

// Suma sobre todos los dígitos finales $T(n) = \sum f(n, d)$

total = 0

para d = 0 hasta 9

total += dp[n][d]

return total

Complejidad: hay $n \times 10$ celdas, cada una calculada en tiempo constante (máximo 5 vecinos). Total: **O(n)** en tiempo, **O(n)** en espacio (o **O(1)** espacio si sólo guardamos la fila anterior).

